



An-Najah National University
Faculty of Engineering and IT

جامعة النجاح الوطنية
كلية الهندسة وتكنولوجيا المعلومات

Distributed Operating Systems

10636456

2nd Semester | 2025-2026

Bazar-com Project | Part 2

Roaa Bzoor	12218040
Yaman Abo Asal	12240055

Cache Consistency

Without Cache:

We measured the average response time for searching 10 different book queries when the cache was disabled. Each request was forwarded directly from the frontend service to the catalog service without storing any previous results. The average response time was higher because every request required inter-service communication between the frontend and catalog containers.

With Cache:

After enabling caching in the frontend service, we repeated the same 10 search queries. The first request for each topic was forwarded to the catalog service, but subsequent requests were served directly from the cache without contacting the catalog service. This significantly reduced the response time, especially for repeated queries.

```
18  
19 ▶ cache = {}  
20 ▶ search_cache = {}  
21 ▶
```

```
if topic in search_cache:  
    print("SEARCH CACHE HIT", flush=True)  
    return jsonify(search_cache[topic])  
  
print("SEARCH CACHE MISS", flush=True)
```

```
if id in cache:  
    print("CACHE HIT", flush=True)  
    return jsonify(cache[id])  
  
print("CACHE MISS", flush=True)
```

```
if id in cache:  
    print("INVALIDATING INFO CACHE", flush=True)  
    del cache[id]  
  
print("INVALIDATING SEARCH CACHE", flush=True)  
search_cache.clear()
```

As explained in the frontend service, when requesting information for a specific book, the system first checks if the data exists in the cache. If it is found, it is returned directly. If not, the request is forwarded to the catalog service to retrieve the data, and then it is stored in the cache for future use.

To evaluate the performance of the system, the average response time was calculated for both query and purchase operations with and without caching.

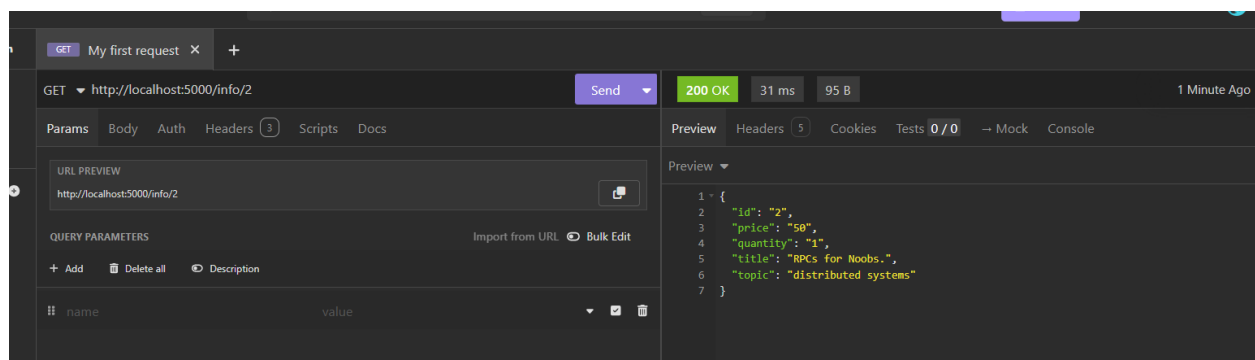
Query Part:

The average response time was measured for 10 search requests without cache and 10 search requests with cache enabled.

Purchase Part:

The average response time was also measured for 5 purchase requests without cache and 5 purchase requests with cache enabled.

With cache:



Output example:

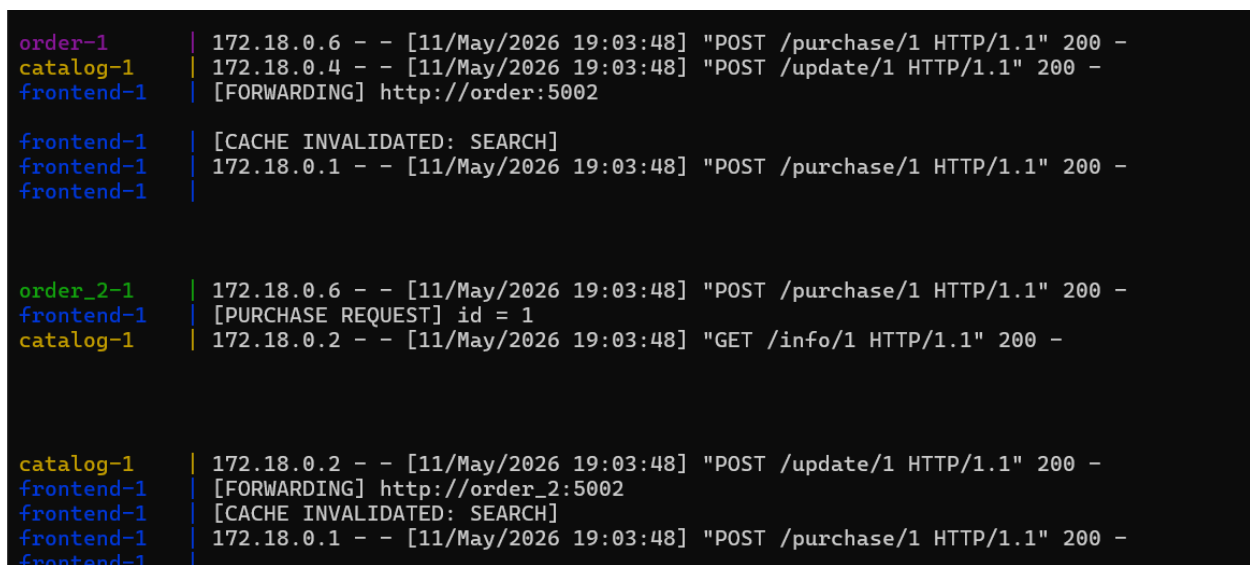
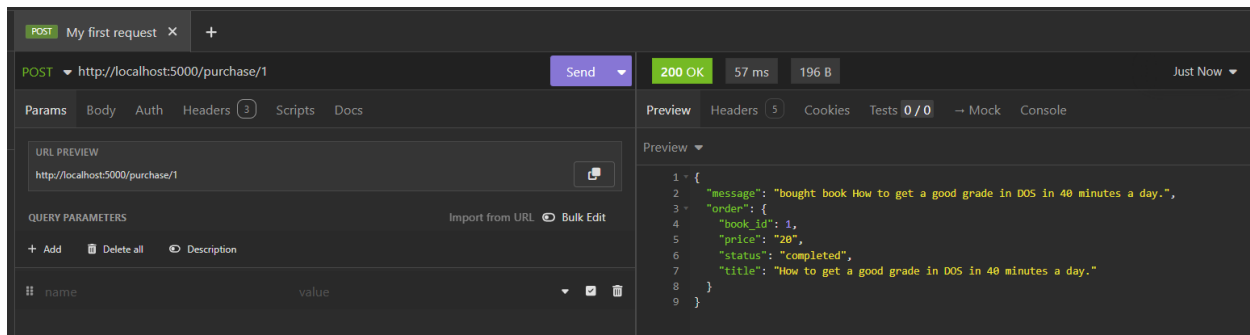
```
catalog-1 | 172.18.0.6 - - [11/May/2026 18:57:25] "GET /info/2 HTTP/1.1" 200 -
frontend-1 | [CACHE MISS]
frontend-1 | [FORWARDING] http://catalog:5001
frontend-1 | [CACHE STORE] id 2 cached
frontend-1 | 172.18.0.1 - - [11/May/2026 18:57:25] "GET /info/2 HTTP/1.1" 200 -
frontend-1 |
frontend-1 | [INFO REQUEST] id = 2
frontend-1 | [CACHE HIT]
frontend-1 | 172.18.0.1 - - [11/May/2026 18:57:31] "GET /info/2 HTTP/1.1" 200 -
```

Average time for 10 searches by id operations: 20.7ms

Performance for caching in search operation: $(63.5 - 20.7)/63.5 * 100\% = 67.4\%$

For Purchase Part:

With cache:



Operation	Avg. With cache	Avg. Without cache	Improvement
Query	20.7	63.5	67.4%
Purchase	148.2	74.2	0.27%

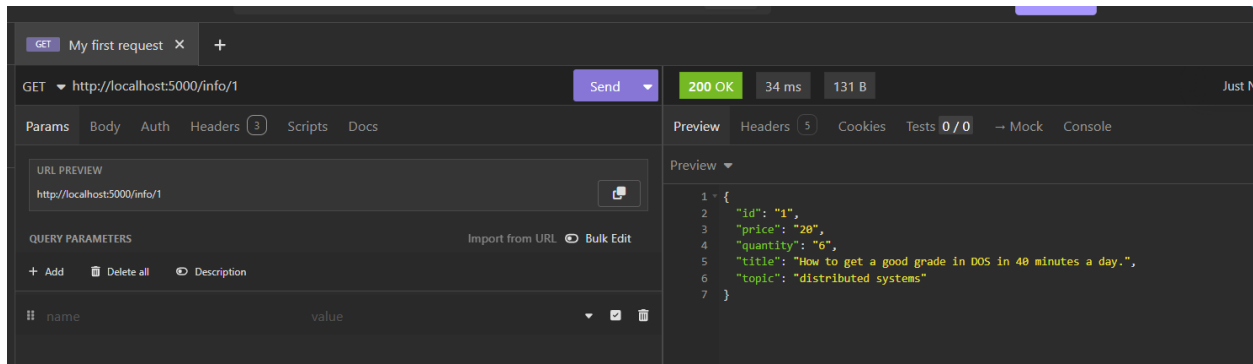
The average response time for purchase operations with caching enabled was slightly higher than without caching. This is because purchase requests are write operations that must be forwarded to the order service, and they also trigger cache invalidation in the frontend.

Therefore, caching introduces a small overhead in purchase operations, but it ensures data consistency. In contrast, caching significantly improves the performance of search and query operations.

To answer this question: Construct a simple experiment that issues orders or catalog updates (i.e., database writes) to invalidate the cache and maintain cache consistency. What is the overhead of cache consistency operations? What is the latency of a subsequent request that sees a cache miss?

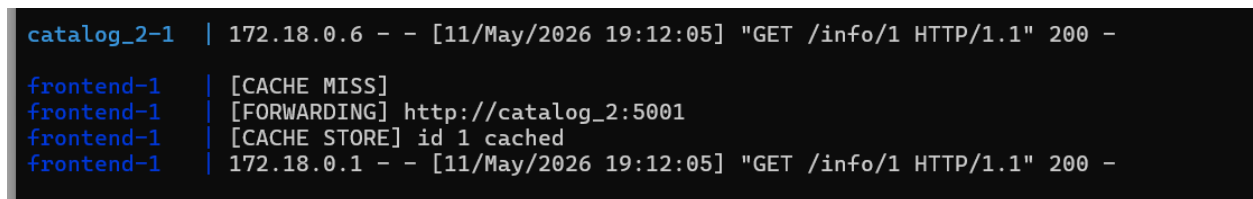
Experiment:

1. Search for book with id = 1 when cache is null:

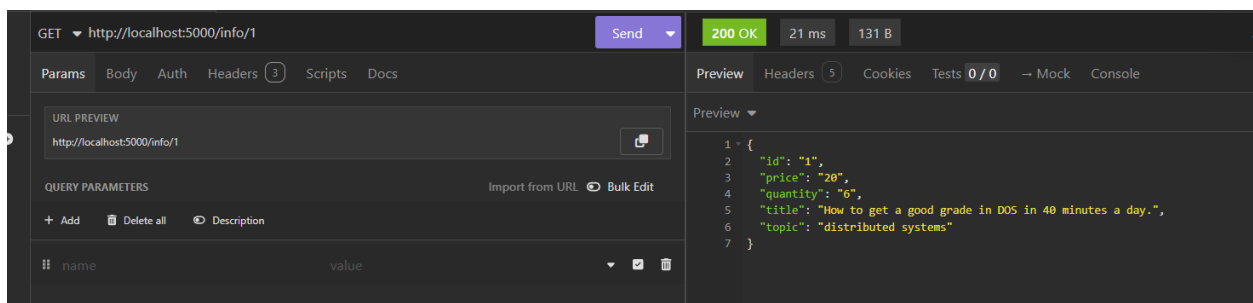


Note: time is 963 ms.

Cache miss =>



2. Search same book again [now it is in cache]:



Note: time is 38 ms.

Cache hit => no going to catalog service

```

frontend-1 | [INFO REQUEST] id = 1
frontend-1 | [CACHE HIT]
frontend-1 | 172.18.0.1 - - [11/May/2026 19:13:27] "GET /info/1 HTTP/1.1" 200 -

```

3. Update operation [patch] on book with id = 4 to see the invalidation.

The screenshot shows a web browser's developer console with the 'Preview' tab selected. The status bar at the top indicates a '200 OK' response, a response time of '37 ms', and a body size of '120 B'. The response is a JSON object with the following structure:

```

1 {
2   "id": 4,
3   "price": "25",
4   "quantity": 8,
5   "title": "Cooking for the Impatient Undergrad.",
6   "topic": "undergraduate school"
7 }

```

The screenshot shows a REST client interface with a GET request to 'http://localhost:5001/info/4'. The 'Body' tab is selected, showing the JSON response:

```

1 {
2   "quantity" : 8
3 }

```

```

frontend-1 | [CACHE MISS]
frontend-1 | [FORWARDING] http://catalog:5001
frontend-1 | [CACHE STORE] id 4 cached
frontend-1 | 172.18.0.1 - - [11/May/2026 19:44:25] "GET /info/4 HTTP/1.1" 200 -
frontend-1 | [INFO REQUEST] id = 4
frontend-1 | [CACHE HIT]
frontend-1 | 172.18.0.1 - - [11/May/2026 19:44:33] "GET /info/4 HTTP/1.1" 200 -

```

Load Balancing

In our system, load balancing is implemented at the frontend layer to distribute client requests across multiple replicated services (catalog and order services).

The frontend maintains a list of available replicas for each service and uses a simple **round-robin algorithm** to forward requests. Each incoming request is sent to the next server in the list, and the index is incremented cyclically.

This mechanism ensures that the workload is evenly distributed between replicas, preventing overloading of a single instance and improving system scalability.

We chose round-robin because it is simple to implement, has minimal overhead, and is suitable for small-scale replicated microservice architectures.

```
server = catalog_servers[catalog_index]
catalog_index = (catalog_index + 1) % len(catalog_servers)
```

```
server = order_servers[order_index]
order_index = (order_index + 1) % len(order_servers)
```

```
catalog-1 | 172.18.0.3 -- [11/May/2026 19:49:50] "POST /update/1 HTTP/1.1" 200 -
frontend-1 | [FORWARDING] http://order:5002

frontend-1 | [CACHE INVALIDATED: SEARCH]
frontend-1 | 172.18.0.1 -- [11/May/2026 19:49:50] "POST /purchase/1 HTTP/1.1" 200 -
frontend-1 |
frontend-1 | [PURCHASE REQUEST] id = 1

catalog-1 | 172.18.0.2 -- [11/May/2026 19:50:03] "GET /info/1 HTTP/1.1" 200 -

v View in Docker Desktop  o View Config  w Enable Watch  d Detach

order_2-1 | 172.18.0.6 -- [11/May/2026 19:50:03] "POST /purchase/1 HTTP/1.1" 200 -
frontend-1 | [FORWARDING] http://order_2:5002
catalog-1 | 172.18.0.2 -- [11/May/2026 19:50:03] "POST /update/1 HTTP/1.1" 200 -
frontend-1 | [CACHE INVALIDATED: SEARCH]
frontend-1 | 172.18.0.1 -- [11/May/2026 19:50:03] "POST /purchase/1 HTTP/1.1" 200 -
```

Docker-aization:

The system was fully containerized using Docker to simplify deployment and ensure consistency across different environments.

Each microservice (frontend, catalog, and order services) runs in an independent container with its own runtime and dependencies, which improves isolation and modularity.

Docker Compose is used to orchestrate the multi-container architecture, where it defines service dependencies, network communication between containers, and port mappings for external access.

In addition, Docker volumes are used to persist the catalog data file across containers. This ensures that both catalog replicas access the same shared dataset, maintaining data consistency even after container restarts.

This containerized architecture makes the system easier to deploy, scale, and manage compared to traditional single-machine setups.

```
docker-compose.yml
1  services:
2  |
3    frontend:
4      build: ./frontend
5      ports:
6        - "5000:5000"
7      depends_on:
8        - catalog
9        - catalog_2
10       - order
11       - order_2
12
13    catalog:
14      build: ./catalog
15      ports:
16        - "5001:5001"
17      volumes:
18        - ./catalog/catalog.csv:/app/catalog.csv
19
20    catalog_2:
21      build: ./catalog
22      ports:
23        - "5003:5001"
24      volumes:
25        - ./catalog/catalog.csv:/app/catalog.csv
26
```

```
app.py frontend  app.py catalog M  docker-compose.yml M X
docker-compose.yml
10     - order
11     - order_2
12
13     catalog:
14         build: ./catalog
15         ports:
16             - "5001:5001"
17         volumes:
18             - ./catalog/catalog.csv:/app/catalog.csv
19
20     catalog_2:
21         build: ./catalog
22         ports:
23             - "5003:5001"
24         volumes:
25             - ./catalog/catalog.csv:/app/catalog.csv
26
27     order:
28         build: ./order
29         ports:
30             - "5002:5002"
31
32     order_2:
33         build: ./order
34         ports:
35             - "5004:5002"
```

How to run the project:

Inside VS code:

- Open terminal in docker folder
- Type: docker compose up --build